

Health-Data Bank:

Database Documentation

Course: CS-4820 — Software Systems Development Project

Date: Jan. 29, 2026

Written By: Alex O'Brien

1. Executive Summary & Architecture

Purpose

This database supports a secure health data management platform designed to store sensitive patient-generated health information while also enabling structured analytics and reporting for researchers and authorized providers.

The system is intended to meet healthcare privacy expectations (HIPAA-aligned principles), including:

- Secure storage of patient health entries
- Controlled access through role-based permissions
- Long-term auditability of all data changes
- Aggregation and anonymization for research use cases

We chose a relational database architecture because health records require strong consistency, clear entity relationships, and enforceable constraints.

Database Engine: PostgreSQL

PostgreSQL was selected as the database engine because it provides:

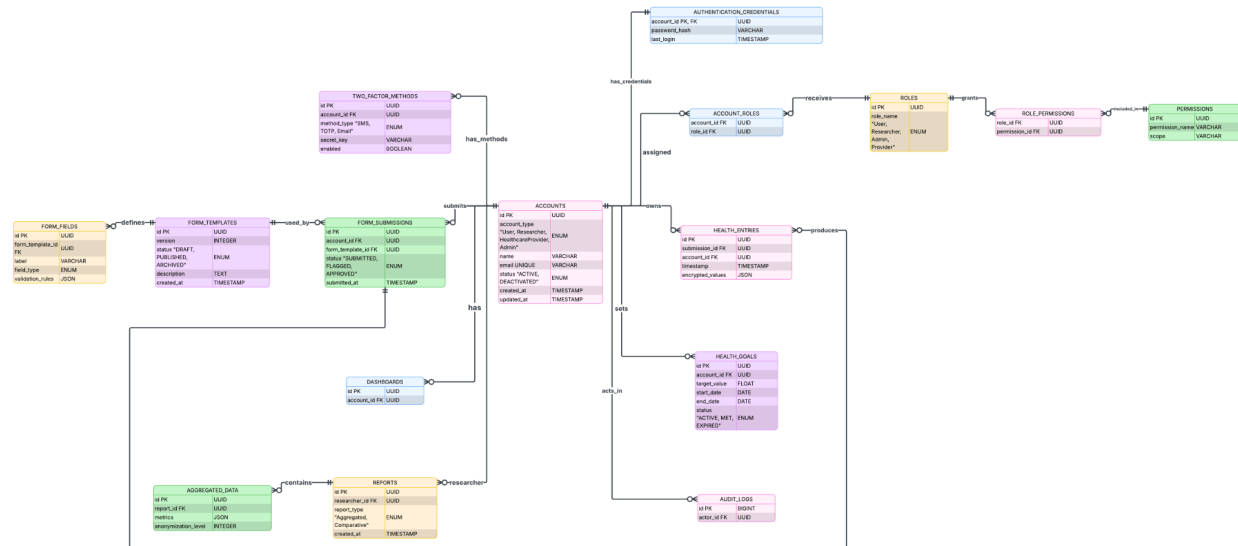
- Strong ACID compliance, ensuring health records are never partially written or corrupted
- Native support for UUID primary keys
- Advanced JSONB functionality, allowing flexible storage
- Extension support (such as encryption and UUID generation)
- Tools for backup, replication, and auditing

Naming Conventions

To ensure consistency and maintainability across the schema, the database follows standardized naming conventions:

- All tables and columns use **snake_case** aligning with PostgreSQL community standards.
- All primary keys are stored as **UUIDs** rather than sequential integers.

2. Entity-Relationship Diagram



3. Data Dictionary

3.1 Account Management & Security

Table 1: accounts *Main registry for all users and system entities.*

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier for all system accounts.

account_type	ENUM	NOT NULL	User, Researcher, HealthcareProvider, Admin.
name	VARCHAR	NOT NULL	Full name of the account holder.
email	VARCHAR	UNIQUE, NOT NULL	Login identifier and contact email.
status	ENUM	DEFAULT 'ACTIVE'	Current state: ACTIVE, DEACTIVATED.
created_at	TIMESTAMP	DEFAULT NOW()	Record creation timestamp.
updated_at	TIMESTAMP	DEFAULT NOW()	Last modification timestamp.

Table 2: authentication_credentials *Secure storage for login verification.*

Column	Type	Constraints	Description
account_id	UUID	PK, FK (accounts.id)	Link to the owner's account.
password_hash	VARCHAR	NOT NULL	Securely hashed password string.

last_login	TIMESTAMP	-	Timestamp of the last successful authentication.
-------------------	-----------	---	--

Table 3: two_factor_methods *Multi-factor authentication settings.*

Column	Type	Constraints	Description
id	UUID	PK	Unique identifier for the 2FA method.
account_id	UUID	FK (accounts.id)	Link to the owner's account.
method_type	ENUM	NOT NULL	SMS, TOTP, Email.
secret_key	VARCHAR	ENCRYPTED	Encrypted secret for code generation.
enabled	BOOLEAN	DEFAULT FALSE	Indicates if the method is currently active.

3.2 Access Control (RBAC)

Table 4: roles *Defined system roles.*

Column	Type	Constraints	Description
--------	------	-------------	-------------

id	UUID	PK	Role identifier.
role_name	ENUM	UNIQUE	User, Researcher, Admin, Provider.

Table 5: permissions *Granular system actions.*

Column	Type	Constraints	Description
id	UUID	PK	Permission identifier.
permission_name	VARCHAR	UNIQUE	Named action (e.g., 'EDIT_REPORTS').
scope	VARCHAR	-	Data scope for the authorization logic.

Table 6: role_permissions *Join table: Maps permissions to roles.*

Column	Type	Constraints	Description
role_id	UUID	FK (roles.id)	The role being granted permission.
permission_id	UUID	FK (permissions.id)	The permission being assigned.

Table 7: account_roles *Join table: Assigns roles to accounts.*

Column	Type	Constraints	Description
account_id	UUID	FK (accounts.id)	The account receiving the role.
role_id	UUID	FK (roles.id)	The role being assigned.

3.3 Forms & Data Collection

Table 8: form_templates *Definitions for data collection forms.*

Column	Type	Constraints	Description
id	UUID	PK	Unique template identifier.
version	INTEGER	NOT NULL	Version control number for templates.
status	ENUM	-	DRAFT, PUBLISHED, ARCHIVED.
description	TEXT	-	Detailed description of the form's purpose.
created_at	TIMESTAMP	DEFAULT NOW()	Timestamp of creation.

Table 9: form_fields *Individual elements within a form template.*

Column	Type	Constraints	Description
id	UUID	PK	Unique field identifier.
form_template_id	UUID	FK (form_templates.id)	The parent template for this field.
label	VARCHAR	NOT NULL	The display name for the field.
field_type	ENUM	NOT NULL	Input data type (Text, Num, Date, etc).
validation_rules	JSON	-	Constraints (regex, min/max, etc).

Table 10: form_submissions *User-submitted form instances.*

Column	Type	Constraints	Description
id	UUID	PK	Unique submission identifier.

account_id	UUID	FK (accounts.id)	The account that submitted the form.
form_template_id	UUID	FK (form_templates.id)	The template used for this submission.
status	ENUM	-	SUBMITTED, FLAGGED, APPROVED.
submitted_at	TIMESTAMP	DEFAULT NOW()	Time the data was received.

3.4 Health Data & Analytics

Table 11: health_entries *The core health data records.*

Column	Type	Constraints	Description
id	UUID	PK	Unique health entry identifier.
submission_id	UUID	FK (form_submissions.id)	Originating form submission.

account_id	UUID	FK (accounts.id)	The data owner (patient).
timestamp	TIMESTAMP	NOT NULL	Time of the actual health event/reading.
encrypted_values	JSON	ENCRYPTED	The actual health data payload.

Table 12: dashboards *User-specific data visualization settings.*

Column	Type	Constraints	Description
id	UUID	PK	Unique dashboard identifier.
account_id	UUID	FK (accounts.id)	The owner of the dashboard view.

Table 13: health_goals *User-defined health targets.*

Column	Type	Constraints	Description
id	UUID	PK	Unique goal identifier.
account_id	UUID	FK (accounts.id)	The account owner setting the goal.

target_value	FLOAT	NOT NULL	The metric to be reached.
start_date	DATE	NOT NULL	Date tracking begins.
end_date	DATE	-	Target completion date.
status	ENUM	-	ACTIVE, MET, EXPIRED.

3.5 Research & Auditing

Table 14: reports *Researcher-generated document metadata.*

Column	Type	Constraints	Description
id	UUID	PK	Unique report identifier.
researcher_id	UUID	FK (accounts.id)	The researcher account that created it.
report_type	ENUM	-	Aggregated, Comparative.
created_at	TIMESTAMP	DEFAULT NOW()	Report generation time.

Table 15: aggregated_data *Processed research data.*

Column	Type	Constraints	Description
id	UUID	PK	Unique record identifier.
report_id	UUID	FK (reports.id)	The parent report for this data.
metrics	JSON	-	The resulting aggregated statistics.
anonymization_level	INTEGER	-	Privacy level applied to this data (1-5).

Table 16: audit_logs *Immutable record of system activity.*

Column	Type	Constraints	Description
id	BIGINT	PK	Auto-incrementing log identifier.
actor_id	UUID	FK (accounts.id)	The account performing the action.
action_type	VARCHAR	NOT NULL	Description of the event or API call.

timestamp	TIMESTAMP	DEFAULT NOW()	Exact time the action occurred.
-----------	-----------	------------------	---------------------------------

4. Security & Privacy Protocols

Health information is highly sensitive, so the database includes multiple layers of privacy protection beyond basic access control.

Anonymization Levels and De-Identification Strategy

The *aggregated_data* table enables population-level health analytics while protecting patient identity through structured anonymization.

To ensure privacy, each aggregated record includes an *anonymization_level* field, which indicates the degree of de-identification applied before the data can be stored or shared.

Minimum Cohort Size Requirement (k-Anonymity Rule)

Before any aggregated data is generated or displayed, the system enforces a **minimum cohort size of 10 participants**.

This means:

- No statistic is stored unless it represents at least 10 individuals
- Queries producing fewer than 10 matching users are suppressed or generalized

This policy exists because small sample sizes create a high risk of re-identification, even if direct identifiers have been removed.

Encryption at Rest

Health entry content is stored as encrypted JSON within the database.

We chose encrypted JSON storage because:

- Health forms evolve frequently (new symptoms, measurements, survey fields)
- A rigid column-per-field model would require constant schema migrations
- JSON allows flexible storage while encryption ensures confidentiality

Anonymization

The aggregated_data table supports research analytics while protecting patient identity through an anonymization_level field and cohort-based privacy constraints.

Anonymization Level Definitions

The system applies anonymization in the following levels:

Level 1 — Direct Identifier Removal

Removes explicit personal identifiers, including:

- Name
- Email address
- Phone number
- Account ID references

At this level, data is no longer directly labeled, but may still contain indirect identifying attributes.

Use case: Internal analytics with minimal masking.

Level 2 — Semi-Identifier Generalization

In addition to Level 1, the system generalizes attributes that could indirectly identify a person, such as:

- Postal code reduced to region/state

- Exact timestamps converted to day-level resolution
- Device identifiers removed

This level prevents linkage attacks where identity could be inferred from combinations of attributes.

Use case: Standard reporting across broad patient populations.

Level 3 — Demographic and Date Obfuscation

In addition to Level 2, the system removes or transforms sensitive demographic markers, including:

- Exact birthdates replaced with age ranges
- Rare diagnosis labels grouped into broader categories
- Small demographic subgroups suppressed

This level significantly reduces the ability to isolate individuals within a dataset.

Use case: Research datasets requiring strong privacy guarantees.

Level 4 — Fully Aggregated, Non-Reversible Analytics

The highest anonymization level contains only statistical summaries, such as:

- Average blood pressure by age group
- Symptom frequency trends across regions
- Condition prevalence over time

At this level:

- No patient-level records remain
- Data cannot be traced back to individuals

- Cohort size enforcement (≥ 10) is mandatory

Use case: External research publication and compliance-safe reporting.

Audit Trail

The audit_logs table records all critical database modifications, including:

- INSERT
- UPDATE
- DELETE

Each log entry includes:

- Which user performed the action
- What table was affected
- When the change occurred
- What values were changed (where permitted)

5. Extensions and Custom Types

PostgreSQL extensions and custom ENUMs were used for consistency and to reduce application-layer complexity.

Extensions

uuid-oss

The uuid-oss extension is enabled to generate UUID primary keys directly inside PostgreSQL.

This was chosen because:

- UUID generation becomes database-native and consistent

- The application does not need to manage ID creation manually
- UUIDs improve security by avoiding predictable identifiers

Example usage:

`uuid_generate_v4()`

Custom ENUM Types

ENUM types were used for fields with a fixed set of valid values.

This improves:

- Data integrity
- Readability
- Constraint enforcement at the database level

Examples include:

account_status

Defines lifecycle state of an account:

- active
- suspended
- closed

This prevents invalid account states from being inserted.

role_name

Defines role-based permissions:

- patient
- provider
- researcher
- admin

Roles are stored as ENUMs to ensure authorization logic is consistent across the system.

method_type

Used for tracking entry submission methods:

- manual
- device_sync
- imported

This allows analytics on how health data is collected without free-text inconsistency.

6. Maintenance & Backup Plans

Healthcare databases require long-term durability and recovery planning.

Backup Schedule

The database is backed up using a multi-tier schedule:

- Nightly full backups to ensure complete recovery points
- Hourly incremental backups to minimize data loss between full snapshots
- Point-in-time recovery (PITR) enabled through WAL archiving

Retention Policy

To balance compliance needs with responsible data minimization, the system applies clear retention limits.

Inactive User Raw Data Retention (1 Year)

Raw health entry data for inactive users is retained for **1 year** before being archived or permanently removed.

This policy exists because:

- Long-term storage of unused identifiable health data increases privacy risk
- Retaining data indefinitely is unnecessary once an account becomes inactive
- Healthcare systems should follow data minimization principles

After 12 months of inactivity, raw entries may be:

- Fully deleted, or
- Converted into anonymized aggregates where appropriate

Audit Log Retention (2 Years)

Audit logs are retained for **2 years** before being archived or removed.

This timeframe was selected because:

- Logs are most useful for recent compliance reviews and incident response
- Long-term retention increases storage costs and sensitive metadata exposure
- A 2-year window provides accountability while limiting unnecessary accumulation

Audit logs remain immutable during retention to preserve forensic integrity.

Ongoing Maintenance

Routine maintenance includes:

- Index monitoring for query performance
- Regular vacuuming and reindexing
- Automated integrity checks on encrypted JSON fields
- Periodic reviews of anonymization enforcement rules

This ensures the database remains performant, compliant, and secure over time.